MODULE 4 – EVENT AND GUI PROGRAMMING IN JAVA

1. EVENT HANDLING IN JAVA

DEFINITION

Event handling in Java is the mechanism used to detect and respond to actions performed by the user or the system.
An action such as clicking a button, pressing a key, or moving the mouse is called an event.
Java uses event classes, listener interfaces, and handler methods to process these events.

EXPLANATION

In a GUI application, the program waits for the user to perform an action. When an action occurs, Java creates an event object and sends it to the appropriate listener.

For example, when a user clicks a button, an ActionEvent is generated and handled using ActionListener.

MAIN PARTS

1. Event Source – Component that generates the event.
2. Event Object – Contains information about the event.
3. Event Listener – Receives the event.
4. Event Handler – Method that performs the required action.

SIMPLE PROGRAM

```
import java.awt.*;
import java.awt.event.*;

class EventDemo extends Frame implements ActionListener {
   Button b;

   EventDemo() {
      b = new Button("Click Me");
      add(b);

      b.addActionListener(this);

      setSize(300, 200);
      setLayout(new FlowLayout());
      setVisible(true);
   }
   public void actionPerformed(ActionEvent e) {
      System.out.println("Button Clicked");
   }
   public static void main(String[] args) {
      new EventDemo();
   }
}
```

OUTPUT

When the button is clicked:

```
Button Clicked
```

IMPORTANT POINTS

- Event handling makes GUI programs interactive.
- ActionListener is commonly used for button actions.
- actionPerformed() handles an action event.
- Event handling is based on the Delegation Event Model.

2. EVENT TYPES IN JAVA

DEFINITION

An event is an occurrence or action that takes place during the execution of a program.
In GUI programming, events are generally generated by user actions such as mouse clicks, key presses, or button selections.
Java provides different event classes and listener interfaces to handle these actions.

COMMON EVENT TYPES

EVENT	MEANING
ActionEvent	Button click or menu selection
MouseEvent	Mouse-related action
KeyEvent	Keyboard action
WindowEvent	Window action
ItemEvent	Selection/deselection of an item
TextEvent	Text modification
FocusEvent	Focus gained or lost

3. MOUSE EVENTS

DEFINITION

Mouse events are events generated when the user performs an action using the mouse.
Examples include clicking, pressing, releasing, entering, or leaving a component.
Java provides the MouseEvent class and listener interfaces to handle these events.

COMMON MOUSE EVENTS

- mouseClicked()
- mousePressed()
- mouseReleased()
- mouseEntered()
- mouseExited()

PROGRAM

```
import java.awt.*;
import java.awt.event.*;

class MouseDemo extends Frame implements MouseListener {

   MouseDemo() {
      addMouseListener(this);

      setSize(300, 200);
      setVisible(true);
   }

   public void mouseClicked(MouseEvent e) {
      System.out.println("Mouse Clicked");
   }

   public void mousePressed(MouseEvent e) {}
   public void mouseReleased(MouseEvent e) {}
   public void mouseEntered(MouseEvent e) {}
   public void mouseExited(MouseEvent e) {}

   public static void main(String[] args) {
      new MouseDemo();
   }
}
```

IMPORTANT POINTS

- MouseEvent belongs to java.awt.event.
- MouseListener is used to handle mouse events.
- mouseClicked() executes when the mouse is clicked.
- Five main methods are provided by MouseListener.

4. KEY EVENTS

DEFINITION

Key events occur when the user interacts with the keyboard.
They are generated when a key is pressed, released, or typed.
Java uses the KeyEvent class and KeyListener interface to handle keyboard actions.

MAIN METHODS

- keyPressed()
- keyReleased()
- keyTyped()

PROGRAM

```
import java.awt.*;
import java.awt.event.*;

class KeyDemo extends Frame implements KeyListener {

   KeyDemo() {
      addKeyListener(this);

      setSize(300, 200);
      setVisible(true);
   }
```

```
    public void keyPressed(KeyEvent e) {
        System.out.println("Key Pressed");
    }

    public void keyReleased(KeyEvent e) {}
    public void keyTyped(KeyEvent e) {}

    public static void main(String[] args) {
        new KeyDemo();
    }
}
```

IMPORTANT POINTS

- KeyEvent represents keyboard events.
- KeyListener handles keyboard events.
- keyPressed() occurs when a key is pressed.
- keyReleased() occurs when a key is released.

5. GUI BASICS

DEFINITION

GUI stands for Graphical User Interface and provides a visual way for users to interact with a program.
A GUI uses components such as buttons, labels, text fields, menus, and windows.
Java provides GUI programming mainly through AWT and Swing.

EXPLANATION

Instead of typing commands, users can interact with graphical objects using a mouse and keyboard.

MAJOR GUI TECHNOLOGIES

1. AWT – Abstract Window Toolkit
2. Swing

EXAMPLES

- Button
- Label
- Text Field
- Check Box
- Radio Button
- List
- Menu
- Window

6. PANELS

DEFINITION

A Panel is a container used to group and organize GUI components in a Java application.
It does not normally appear as an independent window and is placed inside another container.
The Panel class belongs to the AWT package.

SYNTAX

```
Panel p = new Panel();
```

PROGRAM

```
import java.awt.*;
class PanelDemo extends Frame {
   PanelDemo() {
      Panel p = new Panel();
      p.add(new Button("OK"));
      p.add(new Button("Cancel"));
      add(p);

      setSize(300, 200);
      setVisible(true);
   }

   public static void main(String[] args) {
      new PanelDemo();
   }
}
```

IMPORTANT POINTS

- Panel is a container.
- It can contain multiple components.
- It helps organize GUI components.
- It belongs to java.awt.

7. FRAMES

DEFINITION

A Frame is a top-level window used to create a GUI application in AWT.
It provides a window with a title bar and can contain components such as buttons, labels, and text fields.
The Frame class belongs to the java.awt package.

SYNTAX

```
Frame f = new Frame("My Window");
```

PROGRAM

```
import java.awt.*;
class FrameDemo {
   public static void main(String[] args) {
      Frame f = new Frame("My Frame");
      f.setSize(400, 300);
      f.setVisible(true);
   }
}
```

IMPORTANT POINTS

- Frame is a top-level container.
- It can contain GUI components.
- setSize() specifies the size.
- setVisible(true) displays the frame.

8. LAYOUT MANAGERS

DEFINITION

A layout manager controls the arrangement and positioning of GUI components inside a container.
It automatically determines where components should be placed and how much space they should occupy.
Java provides several layout managers such as FlowLayout, BorderLayout, and GridLayout.

MAIN LAYOUT MANAGERS

1. FlowLayout
2. BorderLayout
3. GridLayout

9. FLOWLAYOUT

DEFINITION

FlowLayout arranges components from left to right in a container.
When there is no more space in one row, the components automatically move to the next row.
It is the default layout manager for a Panel.

SYNTAX

```
setLayout(new FlowLayout());
```

PROGRAM

```
import java.awt.*;
class FlowDemo extends Frame {
   FlowDemo() {
      setLayout(new FlowLayout());
      add(new Button("One"));
      add(new Button("Two"));
      add(new Button("Three"));
      setSize(300, 200);
      setVisible(true);
   }
   public static void main(String[] args) {
      new FlowDemo();
   }
}
```

ARRANGEMENT

[One] [Two] [Three]

10. BORDERLAYOUT

DEFINITION

BorderLayout divides a container into five regions for placing components.
The five regions are North, South, East, West, and Center.
It is commonly used when different components need to occupy different areas of a window.

FIVE REGIONS

```
      NORTH
--------------------
WEST |   CENTER | EAST
---------------------
      SOUTH
```

SYNTAX

```
setLayout(new BorderLayout());
```

EXAMPLE

```
add(new Button("North"), BorderLayout.NORTH);
add(new Button("South"), BorderLayout.SOUTH);
add(new Button("East"), BorderLayout.EAST);
add(new Button("West"), BorderLayout.WEST);
add(new Button("Center"), BorderLayout.CENTER);
```

11. GRIDLAYOUT

DEFINITION

GridLayout arranges components in the form of rows and columns.
All cells in the grid have equal size.
It is useful when components need to be arranged systematically like a calculator or keypad.

SYNTAX

```
setLayout(new GridLayout(2, 2));
```

EXAMPLE

```
import java.awt.*;
class GridDemo extends Frame {
   GridDemo() {
      setLayout(new GridLayout(2, 2));
      add(new Button("1"));
      add(new Button("2"));
      add(new Button("3"));
      add(new Button("4"));

      setSize(300, 200);
      setVisible(true);
   }
   public static void main(String[] args) {
      new GridDemo();
   }
}
```

ARRANGEMENT

```
[ 1 ] [ 2 ]
[ 3 ] [ 4 ]
```

12. BUTTON

DEFINITION

A Button is a GUI component used to perform an action when the user clicks it.
It displays a text label such as OK, Submit, Save, or Cancel.
In AWT, the Button class is used to create a button.

SYNTAX

```
Button b = new Button("Click");
```

EXAMPLE

```
Button b = new Button("Submit");
add(b);
```

13. CHECK BOX

DEFINITION

A Check Box is a GUI component that allows the user to select or deselect an option.
Multiple check boxes can normally be selected at the same time.
For example, a student may select Java, Python, and C++ together.

SYNTAX

```
Checkbox c = new Checkbox("Java");
```

EXAMPLE

```
Checkbox c1 = new Checkbox("Java");
Checkbox c2 = new Checkbox("Python");
add(c1);
add(c2);
```

EXAMPLE OUTPUT

☐ Java
☐ Python

14. RADIO BUTTONS

DEFINITION

Radio buttons are GUI components used when the user must select only one option from a group.
In AWT, radio-button behavior can be created using Checkbox and CheckboxGroup.
Selecting one option automatically deselects the previously selected option.

PROGRAM

```
import java.awt.*;
class RadioDemo extends Frame {
   RadioDemo() {
      CheckboxGroup group = new CheckboxGroup();
      Checkbox male =
         new Checkbox("Male", group, false);
      Checkbox female =
         new Checkbox("Female", group, false);
      add(male);
      add(female);
```

```
        setLayout(new FlowLayout());
        setSize(300, 200);
        setVisible(true);
    }
    public static void main(String[] args) {
        new RadioDemo();
    }
}
```

OUTPUT

( ) Male
( ) Female

15. LABEL

DEFINITION

A Label is a GUI component used to display text that normally cannot be edited by the user.
It is commonly used to identify other components such as text fields and buttons.
For example, “Name” can be displayed beside a text field.

SYNTAX

```
Label l = new Label("Name");
```

EXAMPLE

```
Label l = new Label("Enter Name:");
add(l);
```

16. TEXT FIELD

DEFINITION

A Text Field is a GUI component used to accept a single line of text from the user.
It is commonly used for entering information such as name, username, age, or password.
In AWT, the TextField class is used to create a text field.

SYNTAX

```
TextField t = new TextField(20);
```

EXAMPLE

```
Label l = new Label("Name");
TextField t = new TextField(20);

add(l);
add(t);
```

OUTPUT

Name: [_____]

17. TEXT AREA

DEFINITION

A Text Area is a GUI component used to enter or display multiple lines of text.
It provides a larger area than a Text Field and is useful for paragraphs, comments, or addresses.
The TextArea class belongs to AWT.

SYNTAX

```
TextArea t = new TextArea(5, 30);
```

EXAMPLE

```
TextArea t = new TextArea(5, 30);
add(t);
```

18. COMBO BOX

DEFINITION

A Combo Box is a GUI component that provides a drop-down list of choices.
The user can select one item from the available options.
In Swing, the JComboBox class is used to create a combo box.

SYNTAX

```
JComboBox<String> c = new JComboBox<>();
```

PROGRAM

```
import javax.swing.*;
class ComboDemo {
   public static void main(String[] args) {
      JFrame f = new JFrame("Combo Box");
      String items[] = {"Java", "Python", "C++"};

      JComboBox<String> c =
         new JComboBox<>(items);

      f.add(c);

      f.setSize(300, 200);
      f.setVisible(true);
   }
}
```

OUTPUT

```
[ Java ▼ ]
```

19. LISTS

DEFINITION

A List is a GUI component that displays a collection of items from which the user can select one or more items.
It is useful when several choices must be displayed at the same time.
The AWT List class is used to create a list component.

SYNTAX

```
List l = new List();
```

EXAMPLE

```
List l = new List();

l.add("Java");
l.add("Python");
l.add("C++");

add(l);
```

OUTPUT

```
Java
Python
C++
```

20. SCROLL BARS

DEFINITION

A Scroll Bar is a GUI component used to move through content that cannot be completely displayed in the available area.
It allows the user to move horizontally or vertically using a slider.
AWT provides the Scrollbar class for creating scroll bars.

SYNTAX

```
Scrollbar s = new Scrollbar();
```

EXAMPLE

```
Scrollbar s = new Scrollbar(
   Scrollbar.VERTICAL);
add(s);
```

IMPORTANT POINTS

- Used for scrolling.
- Can be horizontal or vertical.
- Belongs to AWT.
- It generates adjustment events.

21. SLIDERS

DEFINITION

A Slider is a GUI component that allows the user to select a value from a continuous range.
The user moves a small knob along a line to increase or decrease the value.
Swing provides the JSlider class for creating sliders.

SYNTAX

```
JSlider s = new JSlider(0, 100);
```

EXAMPLE

```
JSlider s = new JSlider(0, 100);
```

EXAMPLE

```
0 ----|--------- 50 --------|---- 100
          ▲
        Slider
```

USES

- Volume control
- Brightness
- Speed
- Percentage selection

22. WINDOWS

DEFINITION

A Window is a top-level GUI container that represents an area on the screen where components can be displayed.
It can be used to create an independent window or as a base for other GUI containers.
Java AWT provides the Window class for this purpose.

SYNTAX

```
Window w = new Window(frame);
```

IMPORTANT POINTS

- Window is a top-level container.
- It does not normally have a title bar like a Frame.
- It can contain GUI components.
- It belongs to AWT.

23. MENUS

DEFINITION

A Menu is a graphical component that provides a list of commands or options to the user.
Menus are commonly placed at the top of a window and contain menu items.
Java AWT provides MenuBar, Menu, and MenuItem classes for creating menus.

MAIN COMPONENTS

MenuBar
↓
Menu
↓
MenuItem

PROGRAM

```
import java.awt.*;

class MenuDemo extends Frame {

   MenuDemo() {

      MenuBar mb = new MenuBar();

      Menu file = new Menu("File");

      MenuItem open = new MenuItem("Open");
      MenuItem save = new MenuItem("Save");

      file.add(open);
      file.add(save);

      mb.add(file);

      setMenuBar(mb);

      setSize(400, 300);
      setVisible(true);
   }

   public static void main(String[] args) {
      new MenuDemo();
   }
}
```

OUTPUT

File
├── Open
└── Save

24. DIALOG BOX

DEFINITION

A Dialog Box is a small window used to display information or obtain a response from the user.
It is generally used for messages, warnings, confirmations, or input.
Swing provides JOptionPane to create common dialog boxes easily.

COMMON DIALOG TYPES

1. Message dialog
2. Confirm dialog
3. Input dialog

EXAMPLE

```
import javax.swing.*;
class DialogDemo {
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(
            null,
            "Welcome to Java");
    }
}
```

OUTPUT

```
+-------------------------+
| Welcome to Java         |
|                         |
|         [ OK ]          |
+-------------------------+
```

25. APPLET

DEFINITION

An Applet is a small Java program designed to run inside an applet-supported environment.
It was traditionally used to create interactive programs that could be embedded in web pages.
An applet generally extends the Applet class and uses methods such as init(), start(), and paint().

BASIC PROGRAM

```
import java.applet.Applet;
import java.awt.*;

public class MyApplet extends Applet {

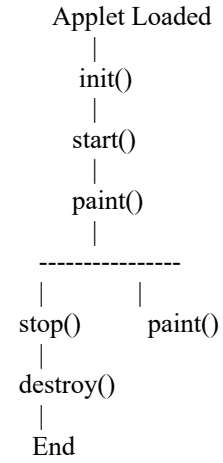
    public void paint(Graphics g) {
        g.drawString("Hello Java", 50, 50);
    }
}
```

26. APPLET LIFE CYCLE

DEFINITION

The Applet Life Cycle describes the sequence of methods called during the execution of an applet.
These methods are controlled by the applet environment rather than directly by the programmer.
The main methods are init(), start(), paint(), stop(), and destroy().

LIFE CYCLE



METHODS

METHOD	PURPOSE
init()	Initializes the applet
start()	Starts or resumes execution
paint()	Displays output
stop()	Stops execution temporarily
destroy()	Releases resources

IMPORTANT POINTS

- init() is called once.
- start() can be called multiple times.
- paint() is used for displaying graphics.
- stop() temporarily stops the applet.
- destroy() is called before the applet is removed.

27. INTRODUCTION TO SWING

DEFINITION

Swing is a Java GUI toolkit used to create graphical applications with rich and flexible components.
It is built on top of AWT and provides lightweight components such as JFrame, JButton, JLabel, and JTextField.
Swing components generally begin with the letter J.

COMMON SWING COMPONENTS

COMPONENT	PURPOSE
JFrame	Main window
JButton	Button
JLabel	Label
JTextField	Single-line text
JTextArea	Multiple-line text
JCheckBox	Check box
JRadioButton	Radio button
JComboBox	Combo box
JList	List
JSlider	Slider

SIMPLE PROGRAM

```
import javax.swing.*;

class SwingDemo {

    public static void main(String[] args) {

        JFrame f = new JFrame("Swing Example");

        JButton b = new JButton("Click Me");

        f.add(b);

        f.setSize(300, 200);
        f.setLayout(new java.awt.FlowLayout());
        f.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);

        f.setVisible(true);
    }
}
```

AWT VS SWING

AWT	SWING
Older GUI toolkit	More advanced GUI toolkit
Uses native components	Mostly lightweight components
Package: java.awt	Package: javax.swing
Fewer components	More GUI components

28. EXCEPTION HANDLING MECHANISM

DEFINITION

Exception handling is the mechanism used to handle runtime problems that occur during program execution.
An exception can interrupt the normal flow of a program if it is not handled properly.
Java provides try, catch, finally, throw, and throws to handle exceptions.

COMMON EXCEPTIONS

- ArithmeticException
- ArrayIndexOutOfBoundsException
- NullPointerException
- NumberFormatException
- IOException

MAIN KEYWORDS

KEYWORD	PURPOSE
try	Contains risky code
catch	Handles the exception
finally	Executes cleanup code
throw	Explicitly generates an exception
throws	Declares possible exceptions

TRY AND CATCH

DEFINITION

The try block contains statements that may produce an exception during program execution.
The catch block is used to handle the exception generated by the corresponding try block.
Using try-catch prevents abnormal termination and allows the program to continue normally.

SYNTAX

```
try {
   // risky code
}
catch(Exception e) {
   // handling code
}
```

PROGRAM

```
class ExceptionDemo {
   public static void main(String[] args) {
      try {
         int a = 10 / 0;
      }
      catch (ArithmeticException e) {
         System.out.println(
            "Cannot divide by zero");
      }
   }
}
```

OUTPUT

Cannot divide by zero

FINALLY BLOCK

DEFINITION

The finally block contains statements that are normally executed whether an exception occurs or not.
It is mainly used for cleanup operations such as closing files or releasing resources.
The finally block is written after the try and catch blocks.

EXAMPLE

```
try {
   int x = 10 / 2;
}
catch (Exception e) {
   System.out.println("Error");
}
finally {
   System.out.println("Program Completed");
}
```

OUTPUT

Program Completed

THROW

DEFINITION

The throw keyword is used to explicitly generate an exception in a Java program.
It is useful when the programmer wants to create an exception based on a particular condition.
It is followed by an exception object.

EXAMPLE

```
throw new ArithmeticException("Invalid value");
```

THROWS

DEFINITION

The throws keyword is used in a method declaration to indicate that the method may generate one or more exceptions.
It informs the calling method that the exception may need to be handled.
It is mainly used with checked exceptions.

SYNTAX

```
void display() throws Exception {
   // statements
}
```

EXCEPTION HANDLING FLOW

